Can Transformers Learn Process Model Structure?

1st Riley Moher
Department of Mechanical and Industrial Engineering
University of Toronto
Toronto, Canada
riley.moher@mail.utoronto.ca

2nd Eldan Cohen
Department of Mechanical and Industrial Engineering
University of Toronto
Toronto, Canada
ecohen@mie.utoronto.ca

***Abstract*—In predictive process monitoring, deep learning methods have been employed to forecast outcomes for ongoing processes, including the next activity prediction task. Recently, these deep-learning based PPM has increasingly embraced the high performance and expressiveness of transformer models. Extending the evaluation framework of Peeperkorn et. al, we present an evaluation framework utilizing variant-based resampling and metrics for fitness, precision, and generalization. Our research confirms that transformers, although slightly less effective than LSTMs in these structural metrics, require fewer parameters and less overfitting countermeasures to better learn process model structure. Moreover, they demonstrate higher accuracy on testing data, even when optimized for these metrics, indicating a promising new direction for explainable predictive process analytics.**

***Index Terms*—process mining, predictive process monitoring, transformer**

I. Introduction

In the landscape of process mining, the predictive and prescriptive modeling methodologies have been gaining popularity. This transition is exemplified in particular by the proliferation of work utilizing deep learning-based techniques, including RNNs [1, 2], and more recently transformers [3, 4, 5], to address predictive monitoring tasks such as remaining time estimation, outcome prediction, and next event prediction. A key concern in these predictive monitoring tasks is whether or not current state-of-the-art modeling architectures, such as transformers [6], effectively "learn" process behavior from potentially incomplete sets of example traces within event logs?

The primary contribution of this work is in extending the framework developed in [1, 2] to evaluate the proficiency of deep learning-based next event prediction techniques, specifically focusing on transformers, in acquiring an understanding of process model structure. Central to our framework is the same variant-based resampling procedure and evaluation metrics aimed at assessing the fitness, precision, and generalization capabilities of learned neural network models as originally introduced in [1, 2].

The extension of this work through transformer models is based primarily on the work in [3], whereby transformers were shown to have state-of-the-art predictive power in the next-activity prediction task. Of particular interest for this analysis, is the tradeoff between expressive and predictive power and the model's ability to generalize unseen behaviour of various process models. Since this analysis is performed on a different architecture, we use similar but different and important hyperparameters in the creation of the evaluation data, including temperature as a new and important focus for discussion.

In addition to the evaluation of transformers, we also more explicitly examine the tradeoffs between predictive accuracy and the ability to capture process model structure. This analysis raises important considerations for predictive models to be put into practice in real predictive process mining (PPM) scenarios.

We find that transformers, while showing excellent predictive capabilities, can learn process model structure similarly to recurrent neural networks while requiring fewer model parameters and requiring less drastic overfitting countermeasures.

The paper is structured as follows: Section II reviews recent work on deep learning techniques in process mining, with a focus on recent advancements using transformers, and discusses their applications in predictive monitoring tasks. Section III details the evaluation framework, including the re-sampling procedure and models used, while also including commentary on the framework itself. Section IV presents the results of our experiments, comparing the performance of transformer models against RNNs in next event prediction tasks. Section V discusses the implications of our findings, particularly the trade-offs between predictive accuracy and model complexity. Finally, Section VI concludes the paper with a summary of our contributions and outlines directions for future research in the field of interpretable predictive process mining.

II. Related Work

In recent years, the field of process mining has begun to undergo a transformation, increasingly embracing deep learning methodologies, especially for predictive process mining tasks. Mirroring broader trends in deep learning, transformers have emerged as a standout architecture for such tasks, owing to their inherent flexibility and capabilities for capturing diverse and complex dependencies within sequential data. However, the rapid proliferation and application of transformers, notably including large language models (LLMs), have motivated the need to address their black-box nature and enhance their interpretability. These principles are equally relevant and vital in the context of process mining.

Prior to the proliferation of transformers within PPM, several different neural network architectures have been proposed for the task. The most popular architecture used was RNNs, with the LSTM being introduced for next activity prediction in [7, 8]. More recently, transformers have been applied for PPM, most notably within ProcessTransformer [3]. In this work, the authors demonstrate the transformer architecture as a novel method of handling predictive process mining tasks including next activity prediction, remaining time prediction, and event time prediction. ProcessTransformer demonstrated that predictive process mining models could achieve state of the art results, even with minimal pre-processing. In [4], the authors apply the BERT language model as the basis for transfer learning in the next activity prediction task. They show that exceptional results, similar to ProcessTransformer and sometimes outperforming it can be achieved, while needing much less time and resources to be dedicated to training, instead relying on fine-tuning of the pre-trained model.

Explainable AI (XAI) techniques in the broader field are designed to demystify the decision-making process of machine learning models for users. Especially in the case of deep learning and large language models, models operate as 'black boxes,' offering little insight into their internal workings. However, better understanding how a model arrives at its decisions can improve trustworthiness and enable increased fairness in decision making. Explainability within AI is usually categorized into intrinsic explanation, whereby the explanation mechanism is baked into the model itself, or post-hoc explanation methods, where explanation is built on top of the model [9]. In the case of this paper, the current evaluation metrics are more similar to post-hoc explanation techniques. While a great deal of work in explainable deep learning-based PPM have also utilized post-hoc explainability techniques [10, 11, 12], there have been several efforts to use intrinsic explainability techniques including specialised attention mechanisms [13].

III. Methodology

A. Evaluation Framework

For our evaluation framework, we base our framework on the one established by Peeperkorn et. al [1, 2], as depicted in Figure 1. This framework works by generating a log containing a random sample of all possible behaviour from a pre-constructed process model, training the model on a majority of behaviours, and selectively holding out a small subset of behaviour for the model to be tested on. The reference log is created by uniformly sampling each unique path in the original process model a specified number of times, in this case we follow the guidance of [2] and set this number to 100 x the number of variants in the model. The entire testing consists of six reference logs, one for each of the process models presented in [1]. The reference log is then resampled to form the training and testing logs, respectively. Unlike the traditional training and testing framework of machine learning methods where training/testing/validation sets are established at the level of individual rows in the data, this is done at the level of variants; all cases in the reference log that pertain to a particular pattern of behaviour (called a variant), or unique path through the process model, are held out. The central idea being that, even when observing a subset of behaviours, the model should still be able to accurately model the process, including unseen behaviours. Once the model has been trained using traces from the training log, a simulated log is generated to approximate the learned behaviours of the model. The simulated log is generated similarly to a decoder in a transformer, taking the encoded start token ([BOS]) as input, and sampling from the transformer model's Softmax output to obtain the next activity in the trace. The newly-generated activity is appended to the trace which is used as subsequent input to the model, with the process being repeated until either the encoded end token is obtained ([EOS]) or the maximum case length for that model is reached.

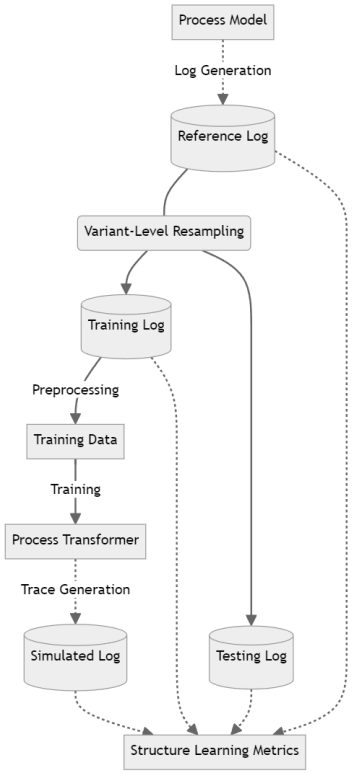

Fig. 1. Evaluation Framework used to test process model structure learning capabilities of the Transformer model trained for next activity prediction

The reference, training, testing, and simulated logs are then used to quantify the degree to which the model is able to *learn* the process model structure. This ability to learn is quantified through the structure learning metrics.

B. Metrics

Ideally, even when trained on a slice of possible behaviours, the model should learn the correct behaviour of the process model, including held out unseen behaviours. What this means in terms of the data generated is that the traces present in the simulated log should match those of the reference log. Furthermore, we want for the model to learn (1) all the behaviours from the training log (2) no behaviours that

are not present in the reference log and (3) all the unseen behaviours present in the testing log. Each of these three goals is reflected in the structure learning metrics, fitness, precision, and generalisation, respectively.

Structure Learning Metric 1. Fitness - The fitness metrics capture the extent to which the model successfully learns and emulates the behaviours contained in the training log. This is measured by the co-occurrence of variants in the training and simulation logs, calculated as:

$$Fitness = \sum_{v \in Vars(Train)} \frac{\text{Min(Count}(v,Sim), \text{Count}(v,Train))}{|Train|}$$

An important observation about the formulation of this metric is that the distribution of occurrences in the simulated log are expected to match that of the training log. We address this in reference to all the metrics after introducing the other metrics.

Structure Learning Metric 2. Precision - Where the fitness metric captured the model's ability to learn the intended behaviour, the precision metric captures the model's ability to learn **only** the intended behaviour.

$$Precision = \sum_{v \in Vars(Sim)} \frac{\text{Min(Count}(v,Sim), \text{Count}(v,Ref))}{|Sim|}$$

Similarly to the formulation of fitness, this metric will also penalize over-representation of correct variants from the reference log.

Structure Learning Metric 3. Generalization - The generalization metric captures the model's ability to emulate correct, but unseen, behaviours. This is measured in terms of the agreement of representation for variants in the simulated log and the testing log.

$$Generalization = \sum_{v \in Vars(Test)} \frac{\text{Min(Count}(v,Sim), \text{Count}(v,Test))}{|Test|}$$

Once again, this variant will also measure the frequency to which unseen variants present in the simulation log agree with the frequency of those variants in the testing log.

Since the reference log was generated via sampling, there will be variability in the representation of each unique variant, but a sufficiently large reference log should mitigate the stochasticity in this calculation. In logs of real-world business processes, however, there would be even greater noise present and variants are not guaranteed to be uniformly distributed. To address this, alternate formulations of the variants are offered in [2] which do not penalize variations in multiplicity but rather model an absolute version of the original metrics. In each of these cases, the counts becomes binary to indicate if the individual variant exists, and the metrics are normalized by the count of total unique variants in each of the training, simulation, and testing logs respectively.

C. Model Architecture

The transformer model used in this work is adapted from the next activity prediction model of ProcessTransformer [3]. Our architecture consists of a positional embedding layer, followed by a transformer block utilizing multi-head dot-product attention, followed by a global average polling layer, dropout layer, a dense layer with a softmax activation and l1 and l2 regularization, and finally, another dropout layer.

D. Model Tuning

Training of the transformer model is subject to various hyperparameters, which will effect not only the predictive capabilities of the model, but also the structure learning metrics. In this work, we seek to address not only whether or not transformers are able to learn process model structure, but to measure to what extent structure learning impacts the traditional predictive performance metrics and vice versa. In order to properly assess this tradeoff, we perform hyperparameter tuning in two ways: for accuracy, and for structure learning. In both cases, we tune the parameters of the model according to Table I:

TABLE I
HYPERPARAMETERS USED FOR TUNING OF THE MODEL

Parameter	Values
Embedding Dimension	$4, 8, 16, 32, 64$
Feedforward Dimension	$4, 8, 16, 32, 64$
Number of Heads	$2, 4, 8, 12, 16$
Dropout Rate	$0.1, 0.2, 0.4, 0.6$
L1 Regularization	$0.0001, 0.001, 0.01, 0.1$
L2 Regularization	$0.0001, 0.001, 0.01, 0.1$
Temperature (only used in structure learning)	$0.05, 0.1, 0.25, 0.75, 1.0, 1.05, 1.25, 3.0$

For the purposes of this work, we also utilize only the results of Model 1 in tuning to effectively utilize the resources available (a modest amount of computation power was used[1]), since performance of the model does vary significantly across models (this is also in line with the tuning strategy of [2].)

Tuning for Accuracy - To accurately reflect hyperparameter tuning that may be expected to be performed in a real predictive process mining scenario, we tune hyperparameters based on a randomly selected validation dataset, a subset of the training log. The training log, prior to being used to train the transformer model, is broken down into all its unique prefixes. For example, the trace $\{A, B, C\}$ would consist of the prefixes: $\{A\}$ and $\{A, B\}$ (ignoring start and end sequence tokens). The validation dataset is created by randomly selecting 20% of the processed training data's rows, or prefixes. The best hyperparameters were chosen based on

[1]Most of the experimentation was performed on CPU, specifically a Ryzen 5 5600G@4.0GHz, with 16GB DDR4@3800MHz RAM

an analysis of the validation data's accuracy, F-Score, Recall and precision. The following parameters chosen based on this analysis are described in Table II.

TABLE II
BEST HYPERPARAMETERS FOR ACCURACY-BASED OPTIMIZATION

Parameter	Chosen Value
Embedding Dimension	16
Feedforward Dimension	16
Number of Heads	2
Dropout Rate	0.2
L1 Regularization	0.0001
L2 Regularization	0.0001

Tuning for Structure-Learning - The purpose of this round of tuning is to examine, in the extreme case, to what extent the structure learning ability comes at the cost of traditionally optimized metrics, like those used in the previous round of tuning. We note this as an extreme because, of course, this kind of post-hoc parameter tuning could never be done in practice, as it requires the synthetic reference log. We first conducted the accuracy-based hyperparameter tuning and used those results as the starting point for this analysis. The final parameters arrived at did not change much, as subtle changes in a small number of parameters made very large impacts on the outcomes. The chosen parameters from this analysis are described in Table III.

TABLE III
BEST HYPERPARAMETERS FOR ACCURACY-BASED OPTIMIZATION

Parameter	Chosen Value
Embedding Dimension	16
Feedforward Dimension	16
Number of Heads	2
Dropout Rate	0.2
L1 Regularization	0.001
L2 Regularization	0.001
Temperature	1.25

IV. RESULTS

In this Section, we present the results of the experiments based on the aforementioned transformer model, under both tuning scenarios. We also compare the results to that of the RNN tested in [1] in terms of the structure learning metrics and accuracy-based metrics including accuracy, F-score, recall and precision.

For each of the six process models developed in [1], we follow a the same testing procedure as the original authors whereby we select a random 20% of variants to be held out from the reference log and form the test log; this is repeated three time with the reported results being an average over the three experiments.[2] The results for this experiment are reported in tables IV, V, firstly for accuracy based optimized parameters, then for structure learning optimized parameters. Note that in these tables, in addition to reporting results for each of the models, we include both the number of total variants in the reference log, as well as the control flow patterns of each of the models to aid in the interpretability of results. The patterns are as follows:

1) PAR - Parallel execution of activites
2) XOR - Exclusive choice between activities
3) LTD - Long-term dependency of activity and a downstream choice
4) IOR - inclusive choice between activites (any number of the activites can be executed, in any order)
5) LOOP - repetition/looping of a subset of activities

TABLE IV
RESULTS COMPARISON BETWEEN RNN AND TRANSFORMER MODELS - ACCURACY-BASED HYPERPARAMETER OPTIMIZATION

RNN Models					
Mdl.	**Pttn.**	**Vars.**	**Fitness**	**Precision**	**Generalisation**
1	PAR	120	0.85 ± 0.05	0.87 ± 0.03	0.00 ± 0.02
2	XOR	128	0.89 ± 0.04	0.89 ± 0.06	0.65 ± 0.08
3	XOR+LTD	128	0.90 ± 0.04	0.87 ± 0.03	0.03 ± 0.09
4	IOR	64	0.86 ± 0.06	0.85 ± 0.04	0.56 ± 0.11
5	PAR	126	0.92 ± 0.05	0.90 ± 0.06	0.08 ± 0.18
6	LOOP	27	0.85 ± 0.06	0.84 ± 0.04	0.66 ± 0.17
Transformer Models					
Mdl.	**Pttn.**	**Vars.**	**Fitness**	**Precision**	**Generalisation**
1	PAR	120	0.85 ± 0.07	0.82 ± 0.06	0.69 ± 0.08
2	XOR	128	0.80 ± 0.02	0.79 ± 0.02	0.77 ± 0.05
3	XOR+LTD	128	0.58 ± 0.17	0.57 ± 0.16	0.56 ± 0.16
4	IOR	64	0.86 ± 0.02	0.84 ± 0.02	0.75 ± 0.04
5	PAR	126	0.68 ± 0.06	0.62 ± 0.07	0.42 ± 0.14
6	LOOP	27	0.84 ± 0.02	0.82 ± 0.04	0.75 ± 0.12

TABLE V
RESULTS COMPARISON BETWEEN RNN AND TRANSFORMER MODELS - STRUCTURE-LEARNING HYPERPARAMETER OPTIMIZATION

RNN Models					
Mdl.	**Pttn.**	**Vars.**	**Fitness**	**Precision**	**Generalisation**
1	PAR	120	0.89 ± 0.01	0.93 ± 0.00	0.72 ± 0.04
2	XOR	128	0.92 ± 0.01	0.92 ± 0.01	0.89 ± 0.03
3	XOR+LTD	128	0.91 ± 0.00	0.92 ± 0.01	0.85 ± 0.05
4	IOR	64	0.92 ± 0.01	0.92 ± 0.01	0.60 ± 0.13
5	PAR	126	0.83 ± 0.01	0.94 ± 0.02	0.46 ± 0.06
6	LOOP	27	0.90 ± 0.02	0.91 ± 0.00	0.82 ± 0.14
Transformer Models					
Mdl.	**Pttn.**	**Vars.**	**Fitness**	**Precision**	**Generalisation**
1	PAR	120	0.84 ± 0.06	0.83 ± 0.06	0.71 ± 0.07
2	XOR	128	0.83 ± 0.02	0.82 ± 0.02	0.81 ± 0.05
3	XOR+LTD	128	0.59 ± 0.17	0.59 ± 0.17	0.57 ± 0.17
4	IOR	64	0.85 ± 0.02	0.83 ± 0.02	0.75 ± 0.06
5	PAR	126	0.64 ± 0.08	0.59 ± 0.09	0.42 ± 0.15
6	LOOP	27	0.80 ± 0.03	0.81 ± 0.04	0.81 ± 0.10

[2]The code used to produce these results is publicly available at: https://github.com/riley-momo/Can-Transformers-Learn-Process-Model-Structure-

In addition to the structure learning metrics, we also present the accuracy and F-score for the test data for each of the models tested above in Tables VI and VII.

TABLE VI
RESULTS COMPARISON BETWEEN RNN AND TRANSFORMER MODELS - ACCURACY-BASED HYPERPARAMETER OPTIMIZATION

RNN Models				
Mdl.	**Pttn.**	**Vars.**	**Accuracy**	**F-Score**
1	PAR	120	0.82 ± 0.03	0.79 ± 0.03
2	XOR	128	0.79 ± 0.02	0.80 ± 0.04
3	XOR+LTD	128	0.80 ± 0.03	0.79 ± 0.03
4	IOR	64	0.81 ± 0.05	0.80 ± 0.03
5	PAR	126	0.77 ± 0.05	0.76 ± 0.06
6	LOOP	27	0.75 ± 0.06	0.72 ± 0.03
Transformer Models				
Mdl.	**Pttn.**	**Vars.**	**Accuracy**	**F-Score**
1	PAR	120	0.83 ± 0.03	0.80 ± 0.03
2	XOR	128	0.81 ± 0.04	0.80 ± 0.04
3	XOR+LTD	128	0.79 ± 0.03	0.78 ± 0.02
4	IOR	64	0.85 ± 0.05	0.81 ± 0.02
5	PAR	126	0.78 ± 0.06	0.74 ± 0.05
6	LOOP	27	0.79 ± 0.05	0.71 ± 0.01

TABLE VII
RESULTS COMPARISON BETWEEN RNN AND TRANSFORMER MODELS - STRUCTURE-LEARNING HYPERPARAMETER OPTIMIZATION

RNN Models				
Mdl.	**Pttn.**	**Vars.**	**Accuracy**	**F-Score**
1	PAR	120	0.69 ± 0.03	0.63 ± 0.06
2	XOR	128	0.66 ± 0.04	0.64 ± 0.06
3	XOR+LTD	128	0.70 ± 0.05	0.66 ± 0.05
4	IOR	64	0.68 ± 0.07	0.67 ± 0.05
5	PAR	126	0.61 ± 0.07	0.60 ± 0.08
6	LOOP	27	0.62 ± 0.08	0.59 ± 0.05
Transformer Models				
Mdl.	**Pttn.**	**Vars.**	**Accuracy**	**F-Score**
1	PAR	120	0.80 ± 0.03	0.76 ± 0.04
2	XOR	128	0.80 ± 0.01	0.79 ± 0.03
3	XOR+LTD	128	0.78 ± 0.02	0.76 ± 0.05
4	IOR	64	0.80 ± 0.05	0.74 ± 0.03
5	PAR	126	0.76 ± 0.06	0.72 ± 0.04
6	LOOP	27	0.77 ± 0.04	0.69 ± 0.02

V. DISCUSSION

The results of comparing the models results, especially the differences in optimization strategies, demonstrate an interesting difference in properties of deep learning models not typically studied, especially in the case of predictive process mining. Firstly, RNNs show a larger performance gap in both sets of metrics: optimizing for structure learning shows a greater improvement, especially in generalization performance, where RNNs have generalization scores as low as 0.00 as in model 1, improving all the way to above 0.7 in Table V. This is true in the reverse case as well, RNN models exhibit a larger (negative) performance gap in typical predictive metrics (as displayed in tables VI and VII) with accuracy going from above 0.8 in some cases to below 0.7. Conversely, while the Transformer models fail to obtain quite the same high levels of performance in structure learning, specifically in the cases of fitness and precision, it remains much more consistent in both cases.

This is likely an exemplification of difference in the expressive power of each model kind. For RNNs, the model requires a greater degree of overfitting countermeasures to be optimized specifically for the structure learning task. In the default cases of the next activity prediction task, the limits of the expressive capability of the LSTM are nearly reached and the higher performance in the fitness and precision metrics with much lower performance in the generalization task is indicative of the models memorization of traces. In the case where structure learning is optimized, a greater balance of the structure learning metrics are reached with the LSTM, but much to the detriment of the actual next activity prediction task. The overfitting countermeasures become so strong that the model's purpose aligns more with this specific new task of structure learning and fails to learn the next activity prediction task as well. This is in contrast to the Transformer model, where the initial performance of the model in structure learning metrics (Table IV) start out at a more acceptable and consistent level across the various models. Furthermore, Table V shows only a relatively small performance jump of the model in structure learning metrics, indicating that the model was already learning process model structure to a greater extent than was true of the RNN in the accuracy-optimized case. This is also supported by the results of Tables VI and VII, with the Transformer model seeing relatively minimal reduction in next activity prediction performance, requiring a much lesser extent of overfitting countermeasures.

VI. CONCLUSION AND FUTURE WORK

While the world of predictive process mining is undergoing significant changes and improvements, it still requires a good deal of effort dedicated not only to predictive tasks but also to explainability. Through the application and extension of an existing evaluation framework [1, 2], we have demonstrated critical insights into the capabilities of transformer models to learn process model structure, and how these differ from previous techniques involving the application of recurrent neural networks. While RNNs showcased a greater improvement in its understanding of process models, especially in generalization, when optimizing for structure learning, transformers exhibited a more consistent performance across these learning metrics and scenarios. This difference can be attributed to the varying expressive power of each model type, with RNNs requiring stronger overfitting countermeasures to optimize for structure learning, potentially at the expense of the primary next activity prediction task.

Notably, transformers displayed promising potential in learning process model structure with fewer parameters and less susceptibility to overfitting. Despite slightly lower effectiveness in structural metrics compared to RNNs, transformers demonstrated higher accuracy on testing data and maintained greater balance across different optimization scenarios. This

suggests a promising direction for explainable predictive process mining, where transformers could offer valuable insights into process behaviors while maintaining predictive accuracy.

Directions for future work include the development of additional and deeper structure learning metrics as well as alternative applications of transformer models in predictive process mining. While the metrics used in this work work well for assessing a model's capability to learn a variety of process model structures, it would be useful to tie metrics more specifically to the kinds of control flow structures present in the models. For example, considering the degree of parallelism, or comparing performance on models with various levels of nested choices. This would provide a more specific assessment of the model's ability to learn process model structure. Additionally, the learning of process model structure, since in this scenario the models are known a priori, could be explored as a direct task, for example in the transformer scenario, as a sequence-to-graph prediction task. Since the process model can be depicted as a graph (from original petri net or other modelling formalism) this could sever as an interesting alternative to process discovery [14]. Finally, the attention mechanism of transformer models in this case could prove to provide valuable insights into the predictive behaviour of PPM models. Since this is a resource available to transformer models that can also be tweaked and tuned, it could prove valuable to analyze the behaviour of the attention mechanism across the various kinds of predictive models that differ with process model structure. This would prove useful in explaining and, diagnosing, for example, a model's performance gap with one kind of event log vs another, where different process model structures could implicitly be behind each respective log.

References

[1] Jari Peeperkorn, Seppe vanden Broucke, and Jochen De Weerdt. "Can deep neural networks learn process model structure? An assessment framework and analysis". In: *International Conference on Process Mining*. Springer. 2021, pp. 127–139.

[2] Jari Peeperkorn, Seppe vanden Broucke, and Jochen De Weerdt. "Can recurrent neural networks learn process model structure?" In: *Journal of Intelligent Information Systems* 61.1 (2023), pp. 27–51.

[3] Zaharah A Bukhsh, Aaqib Saeed, and Remco M Dijkman. "Processtransformer: Predictive business process monitoring with transformer network". In: *arXiv preprint arXiv:2104.00721* (2021).

[4] Hang Chen, Xianwen Fang, and Huan Fang. "Multi-task prediction method of business process based on BERT and Transfer Learning". In: *Knowledge-Based Systems* 254 (2022), p. 109603.

[5] Bemali Wickramanayake et al. "Building interpretable models for business process prediction using shared and specialised attention mechanisms". In: *Knowledge-Based Systems* 248 (2022), p. 108773.

[6] Ashish Vaswani et al. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[7] Joerg Evermann, Jana-Rebecca Rehse, and Peter Fettke. "Predicting process behaviour using deep learning". In: *Decision Support Systems* 100 (2017), pp. 129–140.

[8] Niek Tax et al. "Predictive business process monitoring with LSTM neural networks". In: *Advanced Information Systems Engineering: 29th International Conference, CAiSE 2017, Essen, Germany, June 12-16, 2017, Proceedings 29*. Springer. 2017, pp. 477–492.

[9] Arun Das and Paul Rad. "Opportunities and challenges in explainable artificial intelligence (xai): A survey". In: *arXiv preprint arXiv:2006.11371* (2020).

[10] Nijat Mehdiyev and Peter Fettke. "Prescriptive process analytics with deep learning and explainable artificial intelligence". In: (2020).

[11] Renuka Sindhgatta et al. "Exploring interpretable predictive models for business processes". In: *Business Process Management: 18th International Conference, BPM 2020, Seville, Spain, September 13–18, 2020, Proceedings 18*. Springer. 2020, pp. 257–272.

[12] Ghada El-Khawaga, Mervat Abu-Elkheir, and Manfred Reichert. "Xai in the context of predictive process monitoring: An empirical analysis framework". In: *Algorithms* 15.6 (2022), p. 199.

[13] Bemali Wickramanayake et al. "Building interpretable models for business process prediction using shared and specialised attention mechanisms". In: *Knowledge-Based Systems* 248 (2022), p. 108773.

[14] Wil MP Van Der Aalst. "Process discovery from event data: Relating models and logs through abstractions". In: *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 8.3 (2018), e1244.